

Case Project: LLM-Powered VoiceBot

Founding Software Engineer @ Merlin

1. Einleitung

Du baust einen **Full-Stack-Prototypen** für die Hansestadt Lüneburg. Über eine Weboberfläche können Bürger mit dem VoiceBot interagieren und Fragen zu städtischen Dienstleistungen & Öffnungszeiten stellen. Dieser ermöglicht die kontinuierliche Audioaufnahme und -ausgabe (Auto-Turn-Voicebot).

- Anruf → Begrüßungsansage → freie Sprachinteraktion
- Intent-Erkennung per **LLM**
- Aufruf von Tools → Antwort **per Audio (TTS)** zurück

Technologien: Python (Backend) LiveKit oder Pipecat, Next.js / TypeScript (Frontend)

Docker: Anwendung muss lokal via Docker ausführbar sein

API Keys: OpenAI & ElevenLabs (hier) - Du kannst aber optional auch andere Services mit eigenen Keys verwenden.

2. Deliverables

- Private Git-Repository mit lokal lauffähigem Code via Docker-Setup
- Kompaktes README.md mit Anleitung zur Ausführung + Architektur / Codestruktur

3. Kernfunktionen

- **Auto-Turn-Voicebot:** kontinuierliche Aufnahme und Audio-Antwort
- **Backend:** Audio → STT → LLM Intent-Erkennung → Tool → TTS-Antwort
- **Frontend:** Einfaches Frontend mit Anruf- und Auflegefunktion + Anzeige der Öffnungszeiten pro Department basierend auf `opening_hours.json`
- **Tools:**
 1. `getOpeningHours`
 2. `queryKnowledgeBase`

4. Tool-Spezifikationen

A) getOpeningHours

- Liefert Öffnungszeiten aus `opening_hours.json` ([link](#))
- Datenstruktur kann nach Belieben angepasst werden

B) queryKnowledgeBase

- Antwort auf Fragen basierend auf den bereitgestellten ~600 Markdown-Dateien ([link](#)) via Retrieval-Augmented-Generation (RAG)

5. Bewertung

- **Funktionalität:** Voice Input, LLM Intent Erkennung, Tool Calls, TTS, Basic Frontend
- **Architektur & Design:** Modularität, Erweiterbarkeit, Error Handling
- **Codequalität:** sauberer Code, Docker lauffähig
- **Pragmatische Entscheidungen:** Tradeoffs & Begründungen
- **Dokumentation:** README

Wichtig: Der Fokus liegt darauf, einen **funktionierenden Prototypen** zu bauen – Perfektion ist nicht erforderlich. Ein großer Teil der Bewertung ergibt sich aus dem anschließenden Interview, in dem du deine Entscheidungen erläuterst, Architektur und Trade-offs begründest und zeigst, dass du holistisch über das System nachgedacht hast. Der Scope ist auf **circa 4–6 Stunden** angesetzt. Um dies realistisch zu schaffen, wird **AI-Accelerated Coding** (z. B. Cursor, Claude Code, etc.) und die Verwendung von Boilerplate-Code erwartet. Die Features sind bewusst nur grob beschrieben – du darfst eigene Entscheidungen treffen und kreative Lösungen entwickeln. Die Bewertung orientiert sich am **Gesamteindruck** des Prototyps sowie an deiner **Begründung** der getroffenen Entscheidungen.